PRD: Hybrid Product Recommendation Engine (MVP)

Oct 29, 2026 · @Master

This PRD defines an MVP for a hybrid product recommender that picks the right method for each shopper: popularity for brand-new users, collaborative filtering for users with history, and text similarity for products with no ratings. It keeps the evaluation, item-based CF, SVD and Streamlit demo from your proposal, adds the cold-start journey and product-text clustering from Paul's project, and runs all of it on one dataset instead of two.

Overview

Problem. Shoppers face too many products. Each single method fails somewhere: popularity ignores taste, collaborative filtering needs history, and text similarity ignores what other people liked.

MVP goal. By 10 November 2026, deliver a working, evaluated recommender with a Streamlit demo app and a documented GitHub repository.

What each source contributes

Source	What this PRD takes from it
Your proposal	One coherent pipeline, user-based and item-based CF, SVD, Precision@K and Recall@K evaluation, Streamlit app, GitHub deliverable
Paul's project	Customer-journey logic (new visitor, first purchase, repeat visitor), popularity for cold start, text-based product similarity
Amazon's public approach	Item-to-item similarity as the backbone for "similar products", with popularity and content features as the cold-start fallback

Limitations this PRD fixes

Every weakness found in the two projects maps to a requirement below.

Limitation	Found in	Fix in this PRD
Dataset not finalized; MovieLens is movies, not products	Your proposal	Amazon Reviews 2023 All_Beauty, with reviews and metadata joined on parent_asin (FR-1); MovieLens only as the fallback
No cold-start plan; "input preferences" cannot work with CF alone	Your proposal	Popularity, content-based and a routing rule (FR-3, FR-6, FR-7)
SVD predicts ratings but you evaluate rankings	Your proposal	Rank unseen items by predicted score; relevant = rating of 4 or more (FR-5, FR-8)
Surprise may fail to install	Your proposal	Install test on day one, with a scikit-learn fallback (Technical requirements)
Accuracy-only metrics hide the filter bubble	Your proposal	Add catalog coverage and list diversity (FR-8)
Popularity list is the same for everyone	Paul's project	Popularity is only the cold-start fallback and a baseline, with a category filter (FR-3)
Two unrelated datasets, so no real end-to-end pipeline	Paul's project	Single catalog end to end (FR-1)
Correlations on few ratings are unreliable	Paul's project	Minimum-ratings and minimum co-rated thresholds (FR-1, FR-4)
Text clusters polluted by HTML leftovers such as "nbsp"	Paul's project	Text-cleaning step and a check of top terms (FR-6)
No evaluation, no method comparison	Paul's project	Train/test split and a comparison table (FR-8)
Notebook only, no usable interface	Paul's project	Modular code plus Streamlit app (FR-9, FR-11)
Part III is mislabeled as item-item CF	Paul's project	Labeled content-based; true item-based CF is a separate model (FR-4, FR-6)
Strategies are shown separately, never switched automatically	Paul's project	Rule-based hybrid router (FR-7)

MVP scope

The MVP is one dataset, four models, one router, one evaluation report and one Streamlit app. Anything that does not serve those five is out.

In scope

- One product catalog with ratings and product text, cleaned and filtered
- Four models: popularity, item-based CF, SVD matrix factorization, content-based text similarity
- A rule-based hybrid router that picks a model from how much history a user has
- Offline evaluation: Precision@K, Recall@K, coverage and diversity, with a cold-start segment
- Streamlit app with three modes: existing user, new user, similar to a product
- A short "why recommended" label on each result
- Saved models, README, requirements file and a limitations section

Out of scope

- Real-time streaming, large-scale or production deployment
- Deep learning or sequential models
- Views, clicks and cart events (ratings only)
- Login, payments, inventory, a database server
- Merging a second dataset
- A/B testing with real users

After the MVP

- Learned blending of model scores instead of rules
- Diversity re-ranking
- Sentence-embedding text features
- Implicit-feedback models
- A FastAPI backend with SQLite

Users and user stories

Three shopper types, taken from Paul's customer journey, plus the evaluator who grades the project.

User	Situation	Story
New visitor	No ratings yet	As a new visitor, I want to see the most popular products, so I can start browsing.
New visitor	Types a search term	As a new visitor, I want products that match my words, so I can find something without a history.
Returning shopper with few ratings	1 to 4 ratings	As a shopper with a few ratings, I want products similar to what I rated, so recommendations feel relevant early.
Returning shopper with history	5 or more ratings	As a regular shopper, I want personalized picks I have not rated, so I discover new products.
Any shopper	Viewing one product	As a shopper, I want similar products and a reason for each, so I can trust the list.
Evaluator	Reviewing the project	As an evaluator, I want a comparison of models on standard metrics, so I can judge which approach works.

Functional requirements

Twelve requirements are Must for the MVP and three are Should, to be cut first if time runs short. Thresholds such as "5 ratings" are defaults to tune, not fixed rules.

ID	Requirement	Priority	Acceptance criteria
FR-1	Data pipeline: load one catalog with ratings and product text, remove duplicates and missing values, keep users and items with at least 3 ratings (1,244 users, 1,463 items, 8,978 ratings on All_Beauty after removing duplicate user-item pairs), save processed files	Must	One script reproduces the processed files; row counts before and after filtering are logged
FR-2	Exploratory analysis: rating distribution, ratings per user and per item, sparsity, top products	Must	Notebook shows each chart with a one-line finding

ID	Requirement	Priority	Acceptance criteria
FR-3	Popularity model: weighted rating = (v/(v+m))·R + (m/(v+m))·C, where R is the item's average rating, v its rating count, C the global mean and m a minimum-count setting	Must	Returns top-N; ranks a 5-star item with 2 ratings below a 4.6-star item with 500
FR-4	Item-based CF: cosine similarity on mean-centered ratings, minimum co-rated users per item pair	Must	Returns similar items for any item and top-N for a user; pairs below the threshold are ignored
FR-5	SVD matrix factorization: rank unseen items by predicted rating	Must	Top-N excludes items the user already rated; trained model saved
FR-6	Content-based model: clean text (strip HTML tags and entities, lowercase, stopwords), TF-IDF, cosine similarity; also answers a free-text search	Must	Top 20 terms per cluster contain zero HTML entities or boilerplate terms, and the stopword-extension log is saved; search for a term returns matching products
FR-7	Hybrid router with thresholds set in one config file (defaults 0 / 1 to 4 / 5 or more; about 461 of 1,244 users have 5 or more training ratings, and 3, 4 and 5 are compared in evaluation): 0 ratings uses popularity; 1 to 4 ratings runs item-based CF and content-based on the items the user rated, then merges the two lists by score (each list scaled to 0 to 1 first); 5 or more uses SVD; items with under 5 ratings use content only	Must	Router returns the model name with each result; a unit test covers each branch; changing a threshold in the config file changes the branch
FR-8	Evaluation module: per-user train/test split, Precision@K, Recall@K, catalog coverage, intra-list diversity, cold-start segment (users with 2 to 4 train ratings, about 783 users); relevant means a test rating of 4 or more	Must	One table compares all models and the hybrid on the same split with K = 10, and reports the number of users in the cold-start segment
FR-9	Streamlit app with three modes: pick a user ID, new user (search term or a few ratings), similar to a product	Must	Each mode returns a list within 2 seconds on a local machine

ID	Requirement	Priority	Acceptance criteria
FR-10	Persist models and processed data so the app loads without retraining	Must	App starts from saved files only
FR-11	Repository: modular code, README, requirements file, limitations section	Must	A fresh clone runs end to end by following the README
FR-12	"Why recommended" label on each result (for example, popular with other shoppers; similar to an item you rated)	Must	Every result shows the model that produced it
FR-13	Text-cluster view: KMeans on TF-IDF with top terms per cluster, kept from Paul's project	Should	Top terms printed per cluster, checked for noise
FR-14	New-user onboarding by rating 3 to 5 shown products	Should	Ratings feed the router and change the results
FR-15	Deploy to Streamlit Cloud	Should	Public link works, or documented local run if memory limits block it

Data requirements

Use the Amazon Reviews 2023 All_Beauty dataset from McAuley Lab: the reviews file All_Beauty.jsonl and the metadata file raw_meta_All_Beauty (parquet). The dataset card lists about 701.5K ratings from 632.0K users on 112.6K products, with MovieLens as the fallback if the data later proves unusable (the join check passed on 30 October). Paul's project paired a ratings-only file with a different company's descriptions, so first check the join: the card says the old asin field is really the parent ID, so join the two files on parent_asin, not asin, and report the share of reviews that find a metadata row.

Field	Comes from	Used by
user_id, parent_asin (item ID), rating (1 to 5), timestamp	All_Beauty.jsonl (reviews)	All collaborative models, evaluation
title, description, features, categories, parent_asin	raw_meta_All_Beauty (parquet)	Content-based model, app display

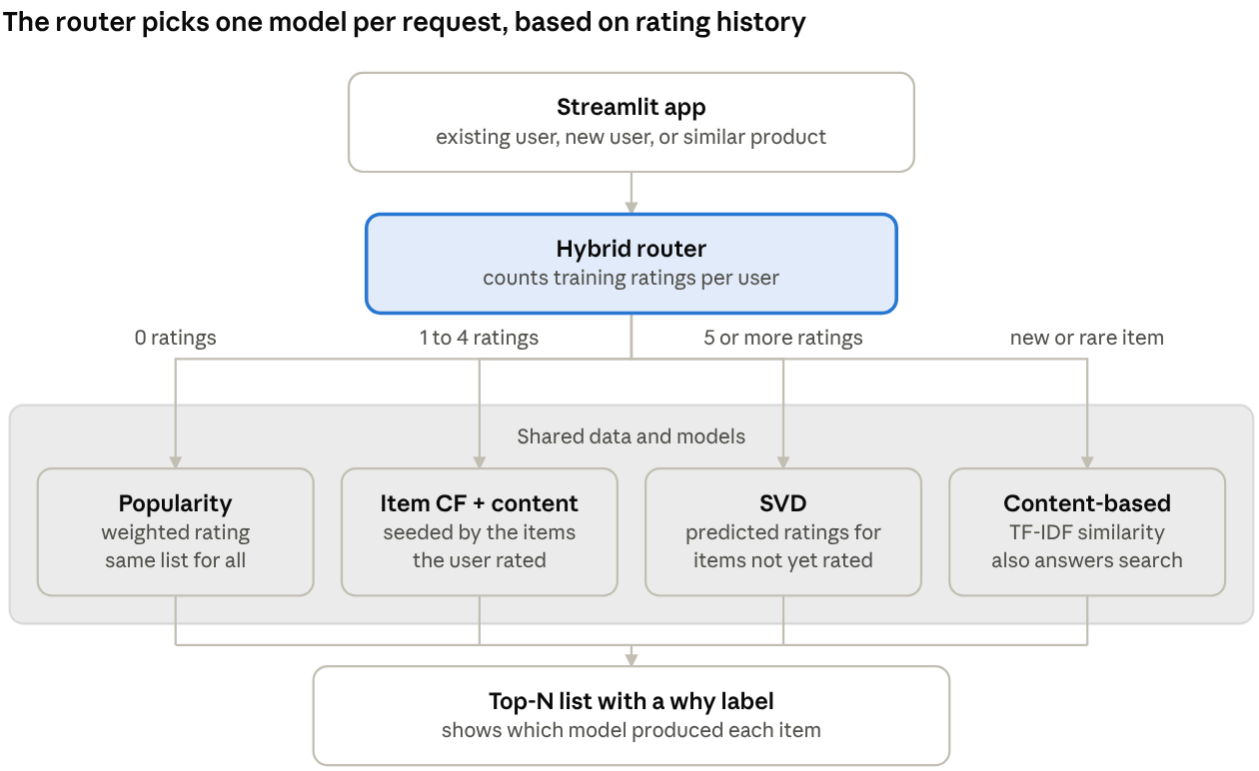
Cleaning rules

- Join reviews to metadata on parent_asin and log how many reviews find no metadata row; drop duplicate user-item pairs (keep the latest) and rows with missing IDs or ratings
- Keep users and items with at least 3 ratings, repeating until no more are removed. All_Beauty has about 1.1 ratings per user, so k = 3 leaves 1,244 users, 1,463 items and 8,978 ratings after removing duplicate user-item pairs (the first join check counted duplicates, which gave 12,219 ratings and 2,087 users). The same filtered data is used for training and testing
- Text: join title, features and description into one string (in the k = 3 set every item has a title, 22.4% have features, 8.4% have a description and none has categories), then strip HTML tags and entities such as " ", lowercase, remove stopwords, then add any boilerplate terms found in the top-term check to the stopword list
- Size: keep the processed data small enough for a free hosting tier; sample users if the catalog is large

Split. Hold out 20% of each user's ratings as test data, by timestamp if available and at random otherwise. A test item counts as relevant when its rating is 4 or more. All models are fit on the training part only.

Architecture and file structure

Every request goes from the app to a router that picks one of four models, and all four read the same processed data and saved files.



request flow · 1 router, 4 models

A user with no ratings gets popularity, a few ratings get item-based CF plus content, and five or more get SVD. A new or rarely rated item always goes to the content-based model.

File structure

```
recommendation-engine/
├── README.md                 # overview, setup, how to run, limitations
├── requirements.txt
├── app.py                    # Streamlit app, three modes (FR-9)
├── data/
│   ├── raw/                  # All_Beauty.jsonl, raw_meta_All_Beauty
parquet; not committed
│   └── processed/            # cleaned ratings and products (FR-1)
├── notebooks/
│   ├── 01_eda.ipynb          # FR-2
│   └── 02_model_comparison.ipynb
├── src/
│   ├── config.py             # thresholds, K, minimum counts (FR-7, FR-8)
│   ├── data_prep.py          # FR-1
│   ├── popularity.py         # FR-3
│   ├── item_cf.py            # FR-4
│   ├── svd_model.py          # FR-5
│   ├── content_model.py      # FR-6, FR-13
│   ├── router.py             # FR-7
│   ├── evaluate.py           # FR-8
```

```
|   ├── eda.py                # FR-2
|   └── try_models.py         # quick check that all models run
├── models/                   # saved models, joblib (FR-10)
├── tests/
|   └── test_router.py
└── reports/
    └── figures/              # charts for the README and report
```

Evaluation and success metrics

The MVP succeeds if personalized models beat the popularity baseline on ranking quality, the hybrid helps users with little history, and the app stays fast. Targets are relative because the absolute scores depend on the dataset.

Metric	Definition (K = 10)	MVP target
Precision@K	Relevant items in the top K, divided by K	Item-based CF or SVD above popularity
Recall@K	Relevant items in the top K, divided by all relevant test items for the user	Item-based CF or SVD above popularity
Catalog coverage	Share of catalog items that appear in at least one user's top K	Personalized models above popularity
List diversity	Average (1 minus cosine similarity) between item text vectors inside one list	Reported for every model, no target
Cold-start segment	Precision@K and Recall@K for users with 2 to 4 training ratings	Hybrid at least equal to the best single model
Text-model reach	Share of items with text for which content-based returns results	100%
App latency	Time from click to results on a local machine	Under 2 seconds

A popularity list can score well on Precision@K because popular items are often liked, so read accuracy together with coverage before naming a winner.

Technical requirements

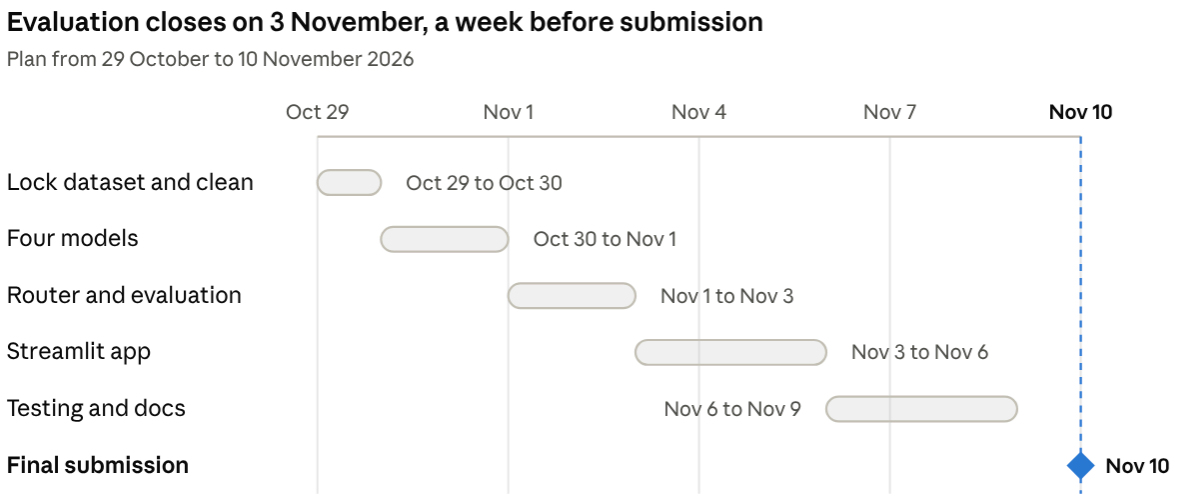
The stack stays close to your proposal, with one change: SVD has a fallback so a failed install cannot block the schedule.

Layer	Choice	Note
Language	Python 3.10 or newer, Jupyter for EDA	Confirm the version on day one
Data	pandas, NumPy, SciPy, PyArrow (reads the parquet metadata file)	Sparse matrices for the ratings
Collaborative models	Own item-based CF with scikit-learn similarity; Surprise SVD	Test `pip install scikit-surprise` on 29 or 30 October
SVD fallback	scikit-learn TruncatedSVD on a mean-centered matrix	Not needed: Surprise installed and is used (confirmed 31 October); kept as a fallback only
Content model	scikit-learn TfidfVectorizer, optional KMeans	HTML cleaning with regular expressions and `html.unescape`
Charts	Matplotlib, Seaborn	EDA and the metrics comparison
App	Streamlit	Load saved models with joblib; cache loading
Versioning	Git and GitHub	Data files not committed, with download steps in the README
Hosting	Local run first; Streamlit Cloud if the files fit its memory limit	FR-15 is Should

A `requirements.txt` with pinned versions is created once the install works.

Timeline

The remaining 12 days to the 10 November submission are ordered so that models come first, then the router and evaluation, then the app.



plan 29 Oct to 10 Nov · 5 phases, 1 deadline

Each phase starts on the day the previous one ends, and the dashed line marks the 10 November deadline.

Gate on 3 November: the model comparison table must exist before app work starts, with at least Precision@K, Recall@K and coverage for popularity, one personalized model and the hybrid. The dataset is locked: All_Beauty with k = 3, decided on 30 October. If the schedule slips, cut in this order: deployment (FR-15), onboarding (FR-14), text-cluster view (FR-13). The router and the evaluation stay, because they are what separates this project from a plain CF notebook.

Risks and open questions

The biggest risks are the dataset and the calendar, and both have a decision date.

Risk	Mitigation
Dataset join gap (closed): many reviews have no metadata row on parent_asin	Closed on 30 October: 100% of reviews and items matched a metadata row on parent_asin
Surprise fails to install	Use the TruncatedSVD fallback and note it in the report
Popularity beats personalized models on Precision@K	Report coverage and the cold-start segment, and explain the result as popularity bias
Very sparse data: even with k = 3 only 1,244 users remain for the collaborative models	Report the user counts, evaluate the hybrid on all users and on a 5-or-more-ratings subgroup, and state it in the limitations

Risk	Mitigation
Time runs out before the app is done	Cut Should items first, then the text-cluster view; never cut evaluation or the router
Files too large for free hosting	Sample users and items, or run locally and record a short demo
Text is still noisy after cleaning	Check top terms per cluster and extend the stopword list

Open questions

- Does the join on parent_asin match most reviews to a metadata row?
- Is a deployed link required for submission, or is a local demo acceptable?
- Has any of the popularity, item-based CF or SVD code already been written, so the schedule can start from it?